LA MINUTE DE CODE



FORMATION: JAVASCRIPT DE A À Z

PARTIE 5

LA MINUTE DE CODE

PLAN DE LA FORMATION

JS

- Le constructeur NEW
- Le spread operator et le rest operator
- Les paramètres par default
- Modules en Javascript
- Les fonctions pures
- Les fonctions d'ordres superieurs
- Utiliser une fonction comme expression
- La programmation orientée objet
- Les classes
- Les méthodes
- Les prototypes

LA MINUTE DE CODE

LE CONSTRUCTEUR NEW

JS

En JavaScript, le constructeur new est utilisé pour créer une nouvelle instance d'un objet à partir d'une fonction constructeur. Les fonctions constructeur sont des fonctions spéciales qui sont utilisées pour créer des objets avec des propriétés et des méthodes.
Voici un exemple simple de création d'un objet avec le constructeur new :

```
function Person(name, age) {
  this.name = name;
  this.age = age;
  this.greet = function() {
    console.log('Bonjour, je m\'appelle '
+ this.name + ' et j\'ai ' + this.age + '
ans.');
  };
}

const john = new Person('John', 30);
john.greet(); // affiche "Bonjour, je
m'appelle John et j'ai 30 ans."
```

LE CONSTRUCTEUR NEW

Dans cet exemple, nous avons créé une fonction constructeur Person qui prend deux arguments : name et age. Nous avons ensuite défini les propriétés name et age pour l'objet créé à partir de la fonction constructeur.

Nous avons également défini une méthode greet() pour l'objet, qui affiche un message de salutation à l'utilisateur en utilisant les propriétés name et age.

Ensuite, nous avons créé une nouvelle instance de l'objet Person en utilisant le constructeur new et en passant les arguments John et 30. Nous avons stocké cette instance dans la variable john.
Enfin, nous avons appelé la méthode greet() de l'objet john pour afficher le message de salutation.

Le constructeur new est une fonctionnalité importante en JavaScript qui permet de créer des objets avec des propriétés et des méthodes en utilisant des fonctions constructeur. En utilisant new, vous pouvez créer de nouvelles instances d'objets avec des propriétés et des comportements spécifiques à chaque instance.

LA MINUTE DE CODE

SPREAD ET REST OPERATOR

JS

En JavaScript, le "reste" (ou "rest" en anglais) et le "spread" sont deux concepts étroitement liés qui sont utilisés pour manipuler des tableaux et des objets de manière pratique et flexible.

Le reste est un opérateur qui permet de capturer tous les arguments restants dans une fonction en les mettant dans un tableau. Par exemple, si vous avez une fonction qui prend trois arguments, mais que vous en passez quatre, vous pouvez utiliser l'opérateur rest pour récupérer le quatrième argument sous forme de tableau.

Voici un exemple :

```
function myFunction(arg1, arg2, ...rest) {
  console.log(arg1); // affiche le premier argument
  console.log(arg2); // affiche le deuxième argument
  console.log(rest); // affiche un tableau contenant les
arguments restants
}

myFunction("a", "b", "c", "d"); // affiche "a", "b", et
["c", "d"]
```

SPREAD ET REST OPERATOR

Le spread, quant à lui, permet de "dérouler" un tableau ou un objet et d'utiliser ses éléments comme des arguments individuels ou des propriétés individuelles. Par exemple, si vous avez deux tableaux et que vous voulez les combiner en un seul, vous pouvez utiliser l'opérateur spread pour ajouter tous les éléments du premier tableau au début du deuxième tableau.

Voici un exemple :

```
const array1 = [1, 2, 3];
const array2 = [4, 5, 6];

const combinedArray = [...array1, ...array2];

console.log(combinedArray); // affiche [1, 2, 3, 4, 5, 6]
```

SPREAD ET REST OPERATOR

De même, si vous avez un objet avec des propriétés et que vous voulez créer un nouvel objet avec ces propriétés, mais avec une ou plusieurs propriétés modifiées, vous pouvez utiliser l'opérateur spread pour copier toutes les propriétés de l'objet original et les remplacer par les nouvelles valeurs.

Voici un exemple :

```
const myObj = {
  foo: 1,
  bar: 2,
  baz: 3
};

const newObj = {
  ...myObj,
  bar: 4
};

console.log(newObj); // affiche { foo: 1, bar: 4, baz: 3 }
```

LA MINUTE DE CODE

PARAMÈTRES PAR DEFAULT

JS

En JavaScript, de nombreux paramètres ont des valeurs par défaut pré-définies. Les paramètres par défaut sont utilisés pour définir une valeur par défaut si une valeur n'est pas fournie pour un paramètre lors de l'appel d'une fonction.

Cela permet aux développeurs de créer des fonctions plus flexibles qui peuvent être appelées avec ou sans certains paramètres.

Par exemple, considérons la fonction suivante qui calcule la surface d'un rectangle :

```
function calculerSurface(longueur, largeur = 1) {
  return longueur * largeur;
}
```

Dans cette fonction, le paramètre longueur est obligatoire et doit être fourni lors de l'appel de la fonction.

Cependant, le paramètre largeur a une valeur par défaut de 1, ce qui signifie que si aucun paramètre n'est fourni pour largeur, la fonction utilisera automatiquement 1 comme valeur par défaut.

Ainsi, la fonction peut être appelée de deux manières différentes :

```
calculerSurface(5, 3); // retourne 15
calculerSurface(5); // retourne 5
```

PARAMÈTRES PAR DEFAULT

Dans le deuxième appel de fonction, la valeur par défaut de largeur est utilisée, car aucun deuxième argument n'a été fourni.

Les paramètres par défaut peuvent également être définis pour les fonctions fléchées :

```
const calculerSurface = (longueur, largeur = 1) =>
longueur * largeur;
```

Les paramètres par défaut sont une fonctionnalité pratique de JavaScript qui permettent aux développeurs de rendre leurs fonctions plus flexibles et plus faciles à utiliser.

LA MINUTE DE CODE

FONCTIONS PURES

JS

En programmation, une fonction pure est une fonction qui ne modifie pas l'état du programme et qui renvoie toujours la même sortie pour une entrée donnée.

En d'autres termes, une fonction pure ne dépend que des entrées qu'elle reçoit et ne modifie pas l'état global de l'application. Les fonctions pures sont considérées comme des éléments clés de la programmation fonctionnelle.

En JavaScript, les fonctions pures sont généralement définies sans effets de bord, c'est-à-dire sans modification de variables globales, sans appels à des fonctions qui ont des effets secondaires et sans interaction avec des entrées/sorties.

Elles sont simplement définies pour prendre une ou plusieurs entrées et renvoyer une valeur basée sur ces entrées.

Voici un exemple de fonction pure en JavaScript :

```
function multiplier(x, y) {
  return x * y;
}
```

FONCTIONS PURES

Dans cet exemple, la fonction multiplier prend deux paramètres, x et y, et renvoie simplement leur produit. Cette fonction est considérée comme pure car elle ne modifie pas l'état global de l'application et elle renvoie toujours la même valeur pour les mêmes entrées.

Les fonctions pures ont plusieurs avantages, notamment :

- Elles sont plus faciles à tester car elles n'ont pas d'effets de bord.
- Elles sont plus faciles à comprendre et à raisonner car elles ont un comportement prévisible et déterministe.
- Elles peuvent être facilement réutilisées dans d'autres parties de l'application car elles ne dépendent que de leurs entrées.

En résumé, les fonctions pures en JavaScript sont des fonctions qui n'ont pas d'effets de bord et qui renvoient toujours la même sortie pour une entrée donnée. Elles sont considérées comme des éléments clés de la programmation fonctionnelle et ont plusieurs avantages, notamment en termes de testabilité, de compréhension et de réutilisabilité.

FOS

En programmation, une fonction d'ordre supérieur est une fonction qui prend une ou plusieurs fonctions en tant que paramètres et/ou qui renvoie une fonction en tant que résultat.

En d'autres termes, une fonction d'ordre supérieur est une fonction qui manipule d'autres fonctions.

En JavaScript, les fonctions d'ordre supérieur peuvent être utilisées pour créer des fonctions plus flexibles et réutilisables, ainsi que pour implémenter des concepts avancés tels que la programmation fonctionnelle.

Voici un exemple simple de fonction d'ordre supérieur en JavaScript qui prend une fonction en tant que paramètre :

```
function appliquerDeuxFois(fonction, valeur) {
  return fonction(fonction(valeur));
}
```

Dans cet exemple, la fonction appliquerDeuxFois prend deux paramètres : une fonction et une valeur.

Elle applique ensuite la fonction deux fois à la valeur et renvoie le résultat final.

Voici un exemple d'utilisation de la fonction appliquerDeuxFois :

```
function doubler(x) {
  return x * 2;
}

appliquerDeuxFois(doubler, 5); // retourne 20
```

Dans cet exemple, nous utilisons la fonction doubler en tant que paramètre de la fonction appliquerDeuxFois.

La fonction appliquerDeuxFois applique la fonction doubler deux fois à la valeur 5, renvoyant finalement 20.

Les fonctions d'ordre supérieur peuvent également renvoyer une fonction en tant que résultat. Voici un exemple simple de fonction d'ordre supérieur qui renvoie une fonction :

```
function creerMultiplicateur(x) {
  return function(y) {
    return x * y;
  }
}
```

Dans cet exemple, la fonction creerMultiplicateur prend un paramètre x et renvoie une fonction qui prend un paramètre y et renvoie le produit de x et y.

Cette fonction peut être utilisée pour créer des fonctions de multiplication personnalisées avec une valeur x pré-définie :

```
const multiplierParCinq = creerMultiplicateur(5);
multiplierParCinq(3); // retourne 15
```

Dans cet exemple, nous utilisons la fonction creerMultiplicateur pour créer une nouvelle fonction appelée multiplierParCinq qui multiplie un nombre par 5. Nous appelons ensuite cette fonction avec une valeur de 3, qui renvoie 15.

En résumé, une fonction d'ordre supérieur est une fonction qui prend une ou plusieurs fonctions en tant que paramètres et/ou qui renvoie une fonction en tant que résultat. Les fonctions d'ordre supérieur peuvent être utilisées pour créer des fonctions plus flexibles et réutilisables, ainsi que pour implémenter des concepts avancés tels que la programmation fonctionnelle.

LA MINUTE DE CODE

MODULES

JS

En JavaScript, les modules sont des unités de code indépendantes qui peuvent être exportées et importées dans d'autres parties de l'application. Les modules permettent de structurer le code en petits blocs modulaires et réutilisables, facilitant ainsi la maintenance et la collaboration entre plusieurs développeurs.

Il existe deux principaux types de modules en JavaScript : les modules ECMAScript et les modules CommonJS.

Les modules ECMAScript sont la norme pour les modules en JavaScript depuis l'introduction de ECMAScript 6 (ES6). Les modules ECMAScript sont utilisés en utilisant les mots clés import et export.

Les modules ECMAScript permettent d'exporter des variables, des fonctions et des classes, et de les importer dans d'autres fichiers. Voici un exemple simple de module ECMAScript :

```
// Dans le fichier module.js
export function doubler(x) {
  return x * 2;
}

// Dans le fichier app.js
import { doubler } from './module.js';
console.log(doubler(5)); // retourne 10
```

Dans cet exemple, nous exportons la fonction doubler depuis le fichier module.js en utilisant le mot clé export. Nous importons ensuite cette fonction dans le fichier app.js en utilisant le mot clé import.

Les modules CommonJS, quant à eux, sont une méthode de module plus ancienne utilisée principalement dans les environnements de serveur Node.js.

Les modules CommonJS utilisent les mots clés require et module.exports pour importer et exporter du code. Voici un exemple simple de module CommonJS :

```
// Dans le fichier module.js
function doubler(x) {
  return x * 2;
}

module.exports = { doubler };

// Dans le fichier app.js
const { doubler } = require('./module.js');
console.log(doubler(5)); // retourne 10
```

Dans cet exemple, nous exportons la fonction doubler depuis le fichier module.js en utilisant le mot clé module.exports. Nous importons ensuite cette fonction dans le fichier app.js en utilisant la fonction require.

En résumé, les modules en JavaScript sont des unités de code indépendantes qui peuvent être exportées et importées dans d'autres parties de l'application.

Les modules ECMAScript et CommonJS sont deux types de modules couramment utilisés en JavaScript.

Les modules permettent de structurer le code en petits blocs modulaires et réutilisables, facilitant ainsi la maintenance et la collaboration entre plusieurs développeurs.

LA MINUTE DE CODE

FONCTION COMME EXPRESSION

JS

En JavaScript, une fonction peut être utilisée comme une expression en l'affectant à une variable ou en la passant comme argument à une autre fonction. Voici quelques exemples pour mieux comprendre :

Affecter une fonction à une variable :

```
const additionner = function(a, b) {
  return a + b;
};
console.log(additionner(2, 3)); // affiche 5
```

Dans cet exemple, nous avons défini une fonction anonyme qui prend deux arguments et les additionne. Ensuite, nous avons affecté cette fonction à la variable additionner. Nous pouvons maintenant appeler cette fonction en utilisant le nom de la variable.

Passer une fonction comme argument à une autre fonction :

```
function calculer(a, b, operation) {
  return operation(a, b);
}

const resultat1 = calculer(2, 3, function(a, b) {
  return a + b;
});

const resultat2 = calculer(5, 3, function(a, b) {
  return a * b;
});

console.log(resultat1); // affiche 5
console.log(resultat2); // affiche 15
```

Dans cet exemple, nous avons défini une fonction calculer qui prend deux arguments et une fonction operation comme troisième argument.

La fonction calculer appelle ensuite la fonction operation en lui passant les deux premiers arguments. Nous avons ensuite appelé la fonction calculer deux fois en passant des fonctions anonymes comme troisième argument pour effectuer deux opérations différentes.

En résumé, pour utiliser une fonction comme une expression en JavaScript, il suffit de l'affecter à une variable ou de la passer comme argument à une autre fonction. Les fonctions peuvent être définies de manière anonyme ou nommée. L'utilisation de fonctions comme expressions peut rendre le code plus flexible et modulaire.

POO

La programmation orientée objet (POO) est un paradigme de programmation qui permet de structurer le code en utilisant des objets qui contiennent des données et des méthodes. En JavaScript, la POO est supportée grâce à la présence d'objets, de classes et de prototypes.

Objets

Un objet en JavaScript est une collection de propriétés qui contiennent des valeurs. Les propriétés peuvent être des méthodes (fonctions) ou des variables (données). Les objets peuvent être créés en utilisant l'opérateur {} ou avec la syntaxe new Object(). Voici un exemple :

```
const personne = {
  nom: "Dupont",
  prenom: "Jean",
  age: 30,
  direBonjour: function() {
    console.log("Bonjour, je m'appelle " + this.nom
+ " " + this.prenom);
  }
};

personne.direBonjour(); // affiche "Bonjour, je
m'appelle Dupont Jean"
```

Dans cet exemple, nous avons créé un objet personne qui contient plusieurs propriétés, dont la méthode direBonjour

LA MINUTE DE CODE

LES CLASSES

JS

Une classe en JavaScript est une "recette" qui permet de créer des objets avec des propriétés et des méthodes communes. Les classes peuvent être créées en utilisant le mot-clé class. Voici un exemple :

```
class Personne {
  constructor(nom, prenom, age) {
    this.nom = nom;
    this.prenom = prenom;
    this.age = age;
  }

  direBonjour() {
    console.log("Bonjour, je m'appelle " + this.nom
+ " " + this.prenom);
  }
}

const personne1 = new Personne("Dupont", "Jean",
30);
personne1.direBonjour(); // affiche "Bonjour, je
m'appelle Dupont Jean"
```

Dans cet exemple, nous avons créé une classe Personne avec un constructeur et une méthode direBonjour. Nous avons ensuite créé un objet personne1 en utilisant le mot-clé new et en passant des arguments au constructeur.

LA MINUTE DE CODE

MÉTHODES

JS

En programmation orientée objet (POO) en JavaScript, les méthodes sont des fonctions qui sont définies à l'intérieur d'une classe.

Elles sont utilisées pour encapsuler le comportement de l'objet et pour permettre l'interaction avec celui-ci.

Voici un exemple simple d'une classe JavaScript avec une méthode :

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  sayHello() {
    console.log(`Hello, my name is ${this.name} and I am
${this.age} years old.`);
  }
}

const person = new Person("John", 30);
person.sayHello(); // affiche "Hello, my name is John and
I am 30 years old."
```

Dans cet exemple, la classe Person contient une méthode sayHello(), qui utilise les propriétés name et age de l'objet Person pour afficher un message de salutation. La méthode est appelée sur une instance de la classe Person en utilisant la notation dot (.).

LA MINUTE DE CODE

PROTOTYPES

JS

En JavaScript, chaque objet a un prototype qui contient des méthodes et des propriétés. Les prototypes permettent de partager des méthodes et des propriétés communes entre plusieurs objets.

Les prototypes peuvent être définis en utilisant la propriété prototype d'une fonction constructeur ou d'une classe. Voici un exemple :

```
function Personne(nom, prenom, age) {
  this.nom = nom;
  this.prenom = prenom;
  this.age = age;
}

Personne.prototype.direBonjour = function() {
  console.log("Bonjour, je m'appelle " + this.nom + " "
+ this.prenom);
};

const personne1 = new Personne("Dupont", "Jean", 30);
personne1.direBonjour(); // affiche "Bonjour, je
m'appelle Dupont Jean"
```

Dans cet exemple, nous avons créé une fonction constructeur Personne avec des propriétés et un prototype contenant la méthode direBonjour. Nous avons ensuite créé un objet personne1 en utilisant la fonction constructeur et appelé la méthode direBonjour.

En résumé, la programmation orientée objet en JavaScript est basée sur la création d'objets, de classes et de prototypes. Les objets sont des collections de propriétés qui contiennent des valeurs, les classes

MÉTHODES

Les méthodes peuvent également prendre des paramètres, comme toute autre fonction. Voici un exemple avec une méthode qui prend un paramètre :

```
class Rectangle {
  constructor(width, height) {
    this.width = width;
    this.height = height;
  }

  getArea() {
    return this.width * this.height;
  }

  resize(newWidth, newHeight) {
    this.width = newWidth;
    this.height = newHeight;
  }
}

const rectangle = new Rectangle(10, 5);
console.log(rectangle.getArea()); // affiche 50
rectangle.resize(20, 10);
console.log(rectangle.getArea()); // affiche 200
```

Dans cet exemple, la classe Rectangle contient deux méthodes : getArea() qui calcule la surface du rectangle et resize() qui permet de changer la taille du rectangle en modifiant les valeurs de width et height.



laminutedecode.io

@LAMINUTEDECODE

